

Jia — Jhonattan IA

Documentación técnica para desarrolladores · v1.0 (Fase 1)

1. Resumen ejecutivo

Jia (Jhonattan IA) es un asistente personal que responde y, en fases posteriores, actúa **en nombre de Jhonattan**. Se alimenta de las conversaciones que captura su dispositivo **Bee Computer** y de su libro *Desarrollo Personal y Superación* (fuente de estilo y valores).

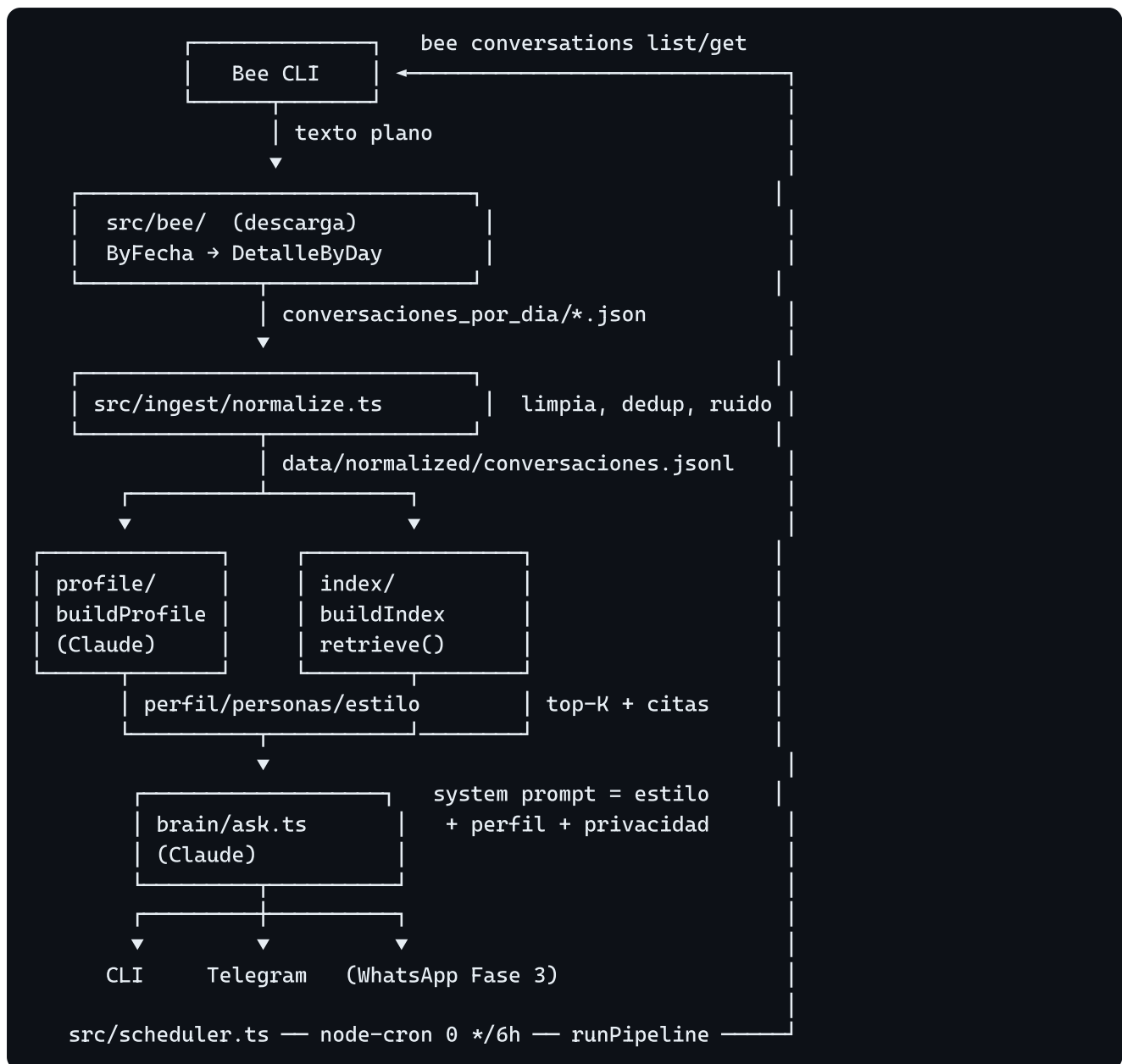
El sistema descarga las conversaciones periódicamente, las limpia y normaliza, construye un **perfil** (quién es, su gente, su voz) y permite hacerle preguntas que responde **como Jhonattan, con citas** a las conversaciones de origen.

Principio rector — Borrador + Aprobación: toda acción con efecto externo (responder un WhatsApp, un pago móvil) se genera como *borrador* y requiere aprobación humana antes de ejecutarse. La Fase 1 documentada aquí es **solo lectura**: ingesta, perfil y Q&A.

Atributo	Valor
Lenguaje	TypeScript (CommonJS, target ES2020)
Runtime	Node.js 18+
Motor IA	Claude (Anthropic) — <code>claude-opus-4-8</code> , <code>claude-haiku-4-5</code>
Fuente de datos	CLI <code>bee</code> (Bee Computer)
Programación	<code>node-cron</code> (cada 6h, configurable)
Estado	Fase 1 completa (typecheck verde)

2. Arquitectura

2.1 Vista de alto nivel



2.2 Decisiones de diseño

- **Proyecto independiente** (C:\Proyectos\Jia), separado del repo `ClienteFeliz` donde nació el descargador de Bee, para tener tooling y ciclo de vida propios.
- **Idempotencia:** `normalize()` deduplica por `id`, por lo que el cron puede solapar días sin generar duplicados. La ventana móvil por defecto es de 2 días.
- **RAG léxico primero:** para ~250 conversaciones, una búsqueda léxica en memoria es suficiente y evita dependencias de vector store. Los embeddings quedan como mejora opcional (ver §9).
- **Perfil como "fuente de verdad":** en lugar de depender de transcripciones ruidosas en cada consulta, se precomputa un perfil estable (`perfil.json`, `personas.json`, `estilo.md`).
- **Privacidad por diseño:** el cerebro tiene reglas explícitas para no exponer temas íntimos a terceros; condición previa para habilitar canales externos (WhatsApp).

3. Estructura del repositorio

```
C:\Proyectos\Jia\  
├─ package.json          scripts y dependencias  
├─ tsconfig.json         TS strict, CommonJS, ES2020  
├─ .env.example          plantilla de claves  
├─ README.md             guía rápida (setup + comandos)  
├─ CONTEXT.md            traspaso de sesión (estado/decisiones)  
├─ docs/  
│   └─ DOCUMENTACION-DESARROLLADORES.md  (este documento)  
├─ src/  
│   ├── scheduler.ts     cron cada 6h → runPipeline  
│   ├── config.ts        carga .env; rutas; ventana/cron de Bee  
│   ├── llm.ts           cliente Anthropic (complete / completeJson)  
│   ├── types.ts         tipos compartidos  
│   ├── cli.ts           consola de preguntas  
│   └─ bee/  
│       ├── ConversacionesByFecha.ts      bee conversations list  
│       ├── ConversacionesDetalleByDay.ts bee conversations get → por_dia  
│       └─ pipeline.ts                    list→detalle→normalize→index  
├─ ingest/  
│   └─ normalize.ts    limpieza + dedup → JSONL  
├─ profile/  
│   ├── buildProfile.ts perfil/personas/estilo (Claude)  
│   └─ personas.seed.ts gente verificada con alias  
├─ index/  
│   └─ buildIndex.ts   búsqueda léxica + retrieve()  
├─ brain/  
│   └─ ask.ts          responde como Jhonattan (Claude)  
├─ telegram/  
│   └─ bot.ts          consola de control privada  
└─ data/  
    └─ (en .gitignore – datos sensibles)  
        ├── profile/libro.md  libro (fuente de estilo)  
        ├── normalized/conversaciones.jsonl  
        └─ index/lexico.json
```

4. Flujo de datos y modelos

4.1 Pipeline de ingesta (`src/bee/pipeline.ts`)

`runPipeline(lookbackDays)` ejecuta en orden:

1. `downloadList(start, end)` — `bee conversations list` paginado; escribe `conversaciones_descargadas/conversaciones_filtradas.json`.
2. `downloadDetailByDay(start, end)` — `bee conversations get <id>` por conversación; agrupa y escribe `conversaciones_por_dia/<fecha>_completo.json` (+ `.txt` legibles).
3. `normalize()` — produce `data/normalized/conversaciones.jsonl`.

4. `buildIndex()` — valida y reporta estadísticas del índice léxico.

4.2 Esquema crudo de Bee (`*_completo.json`)

Array de conversaciones. Campos relevantes:

```
interface RawConversation {
  id: string;
  start_time: string; end_time: string;    // ISO
  state: string;                           // "COMPLETED"
  summary: string; atmosphere: string;
  key_takeaways: string[]; action_items: string[];
  raw_content: string;
  detailed_content?: { utterances?: Utterance[] };
}
interface Utterance {
  speaker: string;           // "Jhonattan" | "Unknown" | ...
  text: string;
  spoken_at: string | null;
  start: number; end: number; // offsets en segundos
}
```

4.3 Esquema normalizado (`conversaciones.jsonl` , una por línea)

```
interface NormalizedConversation {
  id: string;
  fecha: string;           // YYYY-MM-DD (America/Caracas)
  inicio: string; fin: string;
  duracionMin: number;
  summary: string; atmosphere: string;
  key_takeaways: string[]; action_items: string[];
  utterances: Utterance[];
  esRuido: boolean;       // true si parece audio de fondo / medios
}
```

Reglas de limpieza (en `normalize.ts`):

- Marca `esRuido = true` si `summary` / `atmosphere` contienen pistas de medios (background audio, broadcast, podcast, radio, TV, lyrics, commentary, etc.). Por defecto la lectura excluye ruido.
- Colapsa utterances duplicados consecutivos y descarta textos < 2 caracteres.
- Calcula `fecha` y `duracionMin` desde los timestamps; zona horaria America/Caracas (UTC-4).

4.4 Artefactos de perfil

Archivo	Contenido
<code>data/profile/perfil.json</code>	Hechos estables: empresa, cargo, proyectos, valores, rasgos.
<code>data/profile/personas.json</code>	Directorio de gente clave (relación, prioridad, confianza, conflicto, alias).
<code>data/profile/estilo.md</code>	Guía de voz/tono extraída del libro (se inyecta en el system prompt).

5. Referencia de módulos

5.1 `config.ts`

Carga `.env` (vía `dotenv`) y expone `config` (claves, modelos, rutas, parámetros de Bee). `requireConfig(['anthropic' | 'telegram'])` lanza un error claro si falta una clave.

5.2 `llm.ts`

- `getClient()` — cliente Anthropic perezoso.
- `complete(prompt, opts)` — devuelve texto plano.
- `completeJson<T>(prompt, opts)` — parsea JSON (tolera fences ````json`).

5.3 `ingest/normalize.ts`

- `normalize()` — ingesta idempotente a JSONL (clave = `id`).
- `leerNormalizadas(incluirRuido = false)` — lee el JSONL (excluye ruido por defecto).

5.4 `index/buildIndex.ts`

- `tokens(texto)` — minúsculas, sin acentos, sin stopwords ES.
- `retrieve(pregunta, k = 6): Cita[]` — scoring léxico (TF de términos de la query); pondera x2 el resumen y prioriza lo que dijo Jhonattan. Devuelve `{ id, fecha, fragmento, score }`.
- `getConversacion(id)` — recupera una conversación normalizada.
- `buildIndex()` — valida y reporta estadísticas (vocabulario, tokens).

5.5 `profile/buildProfile.ts`

Construye perfil/personas/estilo con Claude sobre los resúmenes + libro. Inyecta `personas.seed.ts` como pista y **garantiza** (merge) que toda persona de la semilla quede en `personas.json` con sus alias (evita confusiones, p. ej. Yul ≠ Dani).

5.6 `brain/ask.ts`

- `ask(pregunta, k = 6): { respuesta, citas }`.

- Construye el system prompt con `estilo.md` + `perfil.json` + `personas.json` + **reglas de privacidad**, recupera contexto con `retrieve()` y consulta `claude-opus-4-8`.

5.7 Interfaces

- `cli.ts` — `npm run ia -- "pregunta"` ; imprime respuesta + fuentes.
- `telegram/bot.ts` — bot privado; **whitelist** de un solo `chat_id` (el dueño).

5.8 `scheduler.ts`

`node-cron` valida `BEE_CRON` y agenda `runPipeline` con timezone America/Caracas. Hace una corrida inicial al arrancar y mantiene el proceso vivo.

6. Configuración (.env)

Variable	Req.	Default	Descripción
<code>ANTHROPIC_API_KEY</code>	Sí (IA)	—	Clave de Claude.
<code>IA_MODEL_REASONING</code>	No	<code>claude-opus-4-8</code>	Modelo de razonamiento/respuesta.
<code>IA_MODEL_CHEAP</code>	No	<code>claude-haiku-4-5-20251001</code>	Modelo barato (clasificación).
<code>TELEGRAM_IA_BOT_TOKEN</code>	Solo bot	—	Token de @BotFather.
<code>TELEGRAM_JHONATTAN_CHAT_ID</code>	Solo bot	—	Único chat autorizado.
<code>BEE_LOOKBACK_DAYS</code>	No	<code>2</code>	Días hacia atrás por corrida.
<code>BEE_CRON</code>	No	<code>cada 6h</code>	Expresión cron de la descarga.

7. Runbook (operación)

```
# Setup
cd C:\Proyectos\Jia
npm install
copy .env.example .env          # completar ANTHROPIC_API_KEY (+ Telegram opcional)

# Backfill inicial (requiere CLI bee autenticado)
npm run bee:download 120        # últimos 120 días

# Generar perfil (consume tokens de Claude)
npm run ia:profile

# Preguntar
npm run ia -- "¿quién es Pablo y cuál fue su último proyecto?"

# Dejar el cron corriendo (descarga + procesa cada 6h)
npm run scheduler

# Producción
npm run build && npm start
```

Script	Acción
<code>npm run scheduler</code>	Cron 6h (corre una vez al iniciar).
<code>npm run bee:download [días]</code>	Pipeline completo una vez.
<code>npm run ia:normalize</code>	Solo normaliza lo descargado.
<code>npm run ia:profile</code>	Genera perfil/personas/estilo.
<code>npm run ia:index</code>	Reconstruye índice.
<code>npm run ia -- " ... "</code>	Q&A por consola.
<code>npm run ia:bot</code>	Bot de Telegram.

8. Seguridad y privacidad

- **Secretos en** `.env` (en `.gitignore`); nunca hardcodear claves. `data/` también está ignorado por contener información personal.
- **Reglas de privacidad** en `brain/ask.ts`: ante temas íntimos/familiares/sensibles, no expone detalles y marca `[ESCALAR_A_JHONATTAN]`. **Obligatorio** antes de habilitar WhatsApp (Fase 3).
- **Telegram con whitelist**: el bot solo responde al `chat_id` del dueño.
- **Calidad de datos**: ~60% de utterances son "Unknown" y hay errores/mezcla ES-EN; por eso el perfil se construye sobre los *resúmenes* (más limpios) y debe revisarse a mano una vez.

9. Extensión y roadmap

Mejoras técnicas previstas

- **RAG con embeddings** (Voyage / vector store local) si la búsqueda léxica se queda corta.
- **Tools de Claude** de solo lectura para reportes de servidores (Fase 4).
- **Cola de aprobación** persistente para borradores (Fases 3/5).

Fases del producto

1. **Fase 1 (esta)**: perfil + Q&A con citas.
2. **Fase 2**: seguimiento de requerimientos (action_items con estado + recordatorios).
3. **Fase 3**: responder WhatsApp en modo borrador (clasificar remitente → redactar → aprobar).
4. **Fase 4**: reportes solo-lectura de servidores.
5. **Fase 5**: acciones simples con aprobación (pago móvil).
6. **Fase 6**: despliegue en la nube.

10. Glosario

- **Bee Computer / CLI** `bee` — dispositivo y CLI que captura y entrega las conversaciones.
- **Utterance** — intervención hablada (speaker + texto + offsets).
- **Normalización** — limpieza/dedup a un JSONL homogéneo.
- **Perfil** — fuente de verdad precomputada sobre Jhonattan.
- **Retrieve / RAG** — recuperación de conversaciones relevantes para dar contexto al modelo.
- **Borrador + Aprobación** — política de seguridad: la IA propone, el humano aprueba.

Documento generado para el equipo de desarrollo de Jia. Versión 1.0 (Fase 1).